

unique_val: a default-on-move type

Document number: P1314R0 [\[latest\]](#)

Date: 2018-10-08

Author: Miguel Ojeda <miguel@ojeda.io>

Project: ISO JTC1/SC22/WG21: Programming Language C++

Audience: LWG, LEWG

Abstract

Modern C++ typically requires writing RAI non-copyable classes at some point or another. In most cases, the class has to wrap a value of some kind (e.g. a pointer, handle, ID, descriptor...) that is meant to be unique and, therefore, needs to default to some value on move. The standard `unique_ptr` class template is the prime example of such a class. This proposal adds a `unique_val` class template to model unique, non-copyable, movable values, simplifying the writing of RAI classes by leveraging the Rule of Zero.

Motivation

Modern C++ is typically written with RAI in mind while taking advantage of the benefits of move semantics brought by C++11. The Rule of Zero summarizes the idea of writing RAI classes that deal with ownership separately, so that the rest of the classes can avoid writing special functions. Therefore, everything that can be done to simplify the writing of resource-owning classes might be a potential improvement.

A very well-known facility in the standard for doing so is `unique_ptr`. Managing heap-allocated memory with it is straightforward. On top of that, its ability to take a custom deleter makes it useful even for cases where a resource is not simply de-/allocated. However, it does not cover some cases:

- The handles/IDs/descriptors of some resources are not necessarily pointers:
 - The handle/ID/descriptor may not even be the size of a pointer.
 - Members such as `operator*` are not appropriate.
 - The `_ptr` part of its name may be misleading.
- No way to specify a different “default”/“invalid” value on move other than `nullptr`.
- While it can be used for “opaque” pointer-type handles/IDs/descriptors, it is needed to assume the opaque type is actually a pointer and remove it from the type; making the code potentially confusing (e.g. [\[1\]](#)[\[2\]](#)).

Extracting both the move semantics and the default-to-`nullptr`-on-move parts of `unique_ptr` into a separate class template `unique_val` allows us:

- To model **opaque pointer-type** handles/IDs/descriptors without assuming they are a pointer (i.e. without having to remove the pointer from the handler type), like the `HWND` handles in the Win32 API.
- To model **non-pointer-type** handles/IDs/descriptors, like the `int` file/socket descriptors in POSIX.
- To model **non-pointer-sized** handles/IDs/descriptors, like `int16_t` IDs in most architectures, `int32_t` IDs in architectures with 64-bit pointers or `int64_t` IDs in architectures with 32-bit pointers.
- To model **“unique” values**, understanding “unique” the same way as `unique_ptr`, i.e. that are supposed to be held by a single owner and destroyed only once.
- To model **values that default on move** to some other value, outside of the scope of RAI classes; including enumerations (`enum` and `enum class`).

Conceptually, `unique_val` can be seen as a more general subset of `unique_ptr`, in the sense that `unique_ptr<T>` can be thought of as a `unique_val<T*>` plus the heap-allocation features.

It can also be used as a better alternative to custom deleters (e.g. `unique_ptr` and `unique_resource` [3]), in case the user prefers to write constructor/destructor pairs (see “Comparison to the proposed `unique_resource`”).

In summary, `unique_val` enables us to model resource ownership with move semantics, both for pointer-types and non-pointer-types; being specially useful for those handles, IDs and descriptors which are supposed to be used as unique values (instead of as unique pointers).

Usage

Assume a C library provides an API like:

```
typedef int FooHandle;
extern "C" FooHandle CreateFoo(/* ... */);
extern "C" void DestroyFoo(FooHandle);
// ...
```

Then `unique_val` can be used to write a non-copyable, movable RAI class like this:

```
class Foo
{
    std::unique_val<FooHandle> id_;

public:
    Foo() : id_(CreateFoo(/* ... */))
    {
        if (not id_)
            throw std::runtime_error("CreateFoo() failed");
    }

    ~Foo()
    {
        if (id_)
            DestroyFoo(id_.get());
    }

    // Foo is already non-copyable, thanks to id_

    // Making Foo movable is not automatic in the current
    // C++ standard, but it is trivial to do so,
    // since id_ already knows how to move itself
    Foo(Foo&&) = default;
    Foo& operator=(Foo&&) = default;
};
```

Now we can trivially use `Foo` as a member of other classes, as usual:

```
class Widget
{
    Foo a_;
    Foo b_;

public:
    Widget() {}

    // Widget is movable and non-copyable, nothing else required
};
```

Comparison to the proposed `unique_resource`

A natural extension to consider for `unique_val` is to add a custom “deleter” (or similar facility) so that an entire RAII class can be modeled directly with `unique_val`, the same way it is done with `unique_ptr` nowadays. In other words, to make `unique_val` the actual RAII class instead of having to write a wrapper class.

This approach was independently taken by P0052 (“Generic Scope Guard and RAII Wrapper for the Standard Library”) [\[3\]](#), which proposes a generic RAII wrapper called `unique_resource` which requires a deleter. It allows to write code such as:

```
auto foo = std::make_unique_resource(  
    CreateFoo(/* ... */),  
    [](FooHandle h) {  
        DestroyFoo(h);  
    }  
);
```

While this approach (using deleters) has its own merits, we argue that the `unique_val` approach (avoiding deleters) is more flexible and simpler at the same time:

- While `unique_resource` allows to quickly define a variable which models a single resource (as shown above), typically reusable RAII types are wanted anyway (to instantiate them in different parts of a project, to provide extra member functions, to contain other members, etc.); defeating that advantage.
- In many cases, it is anyhow needed to write extra logic (e.g. other operations, logging...) while constructing/destructing the RAII class. Using `unique_val` results in code that is arguably easier to follow in the `unique_val` case. In the `unique_resource` case, the code for the destruction of the resource would be in the deleter, while some code would remain in the destructor of the RAII class, separately. On top of that, running code *after* the deleter of the `unique_resource` requires another object being destroyed after the resource itself, e.g. another member variable in the RAII class with a non-trivial destructor; which makes the code even less local.
- `unique_val` encourages writing “symmetrical” constructor/destructor pairs in the RAII class, i.e. the code for constructing the underlying value is in the constructor of the RAII class; and, similarly, the destructing code is in the destructor of the RAII class. In the `unique_resource` case, however, both would be in the constructor of the RAII class (which initializes the deleter).
- `unique_val` is a simpler, “lower-level” primitive that may be used to implement `unique_ptr` and `unique_resource` themselves. In this sense, it makes sense to standardize it as a basic building block (“vocabulary type”).

In summary, we believe `unique_val` is simpler to specify, implement, understand and use than `unique_resource`; while at the same time being more general and leading to easier-to-read application code.

Proposal

This proposal suggests adding a new header, `<unique_val>`, containing a `unique_val` class template and the `make_unique_val` function templates.

unique_val class template

```
namespace std {
    template <class T>
    class unique_val
    {
        T value_; // exposition only

    public:
        using value_type = T;

        explicit unique_val(T = T()) noexcept;

        unique_val(const unique_val<T>&) = delete;
        unique_val(unique_val<T>&&) noexcept;

        unique_val<T>& operator=(const unique_val<T>&) = delete;
        unique_val<T>& operator=(unique_val<T>&&) noexcept;

        auto operator<=>(const unique_val<T>&) = default;

        void swap(unique_val<T>&) noexcept;

        T release() noexcept;
        void reset(T = T()) noexcept;

        T get() const noexcept;
        static constexpr T get_default() noexcept;

        explicit operator bool() const noexcept;
    };

    template <class T>
    void swap(unique_val<T>&, unique_val<T>&) noexcept;

    template <class T>
    struct hash<unique_val<T>>
    {
        std::size_t operator()(const unique_val<T>&) const
noexcept;
    };

    template <class T>
    unique_val<T> make_unique_val(T&&) noexcept;

    template <class T, class... Args>
    unique_val<T> make_unique_val(Args&&...) noexcept;
}
```

The `unique_val` class template is intended to represent a non-copyable, movable unique value. The interface follows that of `unique_ptr`. It provides the following behavior:

- On move assignment and move construction, the moved-from value is reset to the default value.
- `release` resets the underlying value to the default one while returning a copy of the original one.
- `reset` resets the underlying value to the provided one.
- `get` returns a copy of the underlying value.
- `get_default` returns a copy of the default value.
- `operator bool` returns whether the value is different than the default, i.e. typically whether a unique value is contained.

Here, *default value* refers to the default-constructed value of `T`, i.e. `T()`.

Note: `unique_val<T>` will typically be used to wrap an integral, pointer or enumeration type `T`; although other more complex types are possible (e.g. a wrapped built-in type with a different default constructor for cases where `0` or `nullptr` are not the “empty” case).

Note: `unique_val<T>` is intended to have zero runtime-time overhead (in both size and time) with respect to a `T`, except for the resetting operation on move.

Possible extensions

A “default value” template parameter could be added, e.g. `unique_val<T, T DefaultValue>`; so that users can easily specify different default values for built-in types without having to wrap them in another class. This should be implemented as another template with the same name; instead of using a defaulted second template parameter, in order to avoid restricting the usable types to non-type template parameters only.

Some other possible extensions:

- Comparison functions to the naked type `T`.
- `is_default` member function returning whether the value is the same as the default (i.e. the opposite to `operator bool`).
- `operator<<` and `operator>>`.
- `operator*` and `operator->` (like `unique_ptr`). However, they are not pointers.
- Checked vs. unchecked access (e.g. `value` vs `operator*` like `optional`).
- Another class template with runtime default values.
- Custom “deleter” (like `unique_ptr`). However, as explained, we believe it leads to arguably more complex code.

Naming

Several alternatives were considered for the `unique_val` class template:

- `unique_val`: concise, follows `unique_ptr` (including its name length), reflects the most common use case, sounds different than “unique value” when spoken out-loud (which helps disambiguating).
- `unique_value`: ditto, but more verbose and might be confusing when spoken out loud.
- `unique_resource`: too specific, might be too verbose, conflicts with the `unique_resource` proposal [3].
- `unique_handle`: too specific, might be too verbose.
- `handle`: too specific and general at the same time.
- `movable`: too general.
- `resettable`: too general.
- `defaulting`: does not easily reflect the most common use case.
- `default_on_move`: does not easily reflect the most common use case.
- `defaulted`: does not easily reflect the most common use case.

Example implementation

An example implementation can be found at [4].

Acknowledgements

Thanks to Scott Meters, Andrzej Krzemienski, Nevin Liber, Steven Watanabe, Brook Milligan, Richard Hodges, A. Joël Lamotte, Paul A. Bristow for their input!

References

1. `std::unique_ptr`, deleters and the Win32 API — <https://stackoverflow.com/questions/14841396/stdunique-ptr-deleters-and-the-win32-api>
2. Using `std::unique_ptr` for Windows HANDLES — <https://stackoverflow.com/questions/12184779/using-stdunique-ptr-for-windows-handles>
3. P0052R8 - Generic Scope Guard and RAII Wrapper for the Standard Library — <http://open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0052r8.pdf>
4. Example implementation — https://github.com/ojeda/unique_val/tree/master/proposal